



Anil Seth

ZODB, a NoSQL Database

Let's explore how to use ZODB, a NoSQL database, from Python, with an example that stores and retrieves 'album' and 'track' data from the database.

Most of us are accustomed to using a relational database to store large volumes of data. We rarely look for alternatives unless we run into a bottleneck. Even then, you are likely to put in a lot more effort into optimising the database, rather than stepping outside the relational model.

Non-relational databases have been around for many years. When object-oriented programming became popular, a number of object databases were created, but none captured any substantial share of mind. Object-relational-mapping software like Hibernate for Java, SQLAlchemy for Python, and ActiveRecord for Ruby, fulfilled the need of using relational databases within the object-oriented programming paradigm.

SQL is a wonderful tool for arbitrary queries on a relational database. However, you may overestimate the need for it. For example, when dealing with a content-management system, you are more likely to need a keyword-retrieval option, rather than a flexible SQL query. I use a keyword search with Gmail, and I have rarely felt the need to narrow the search to, say, the subject only. Even if I search based on the subject line, I still need a keyword search. I can't recall any need for a search where the use of an index on the subject would have been beneficial—for example, matching a prefix. Hence, a keyword-search tool like Apache Lucene (<http://lucene.apache.org/>) along with any database, whether relational or not, can be a superb solution.

In the last few years, the need for Web-scale databases has increased the interest in 'NoSQL' databases—a misleading term, which is now often interpreted as 'not only SQL' (<http://nosql-database.org/>). One category of such databases is object database management systems (ODBMS), and among them is a native object database for Python—ZODB (<http://www.zodb.org/>). Object databases provide ACID support. They reduce the friction of having to transform objects into relational table rows and vice versa—thus improving the efficiency of accessing and manipulating objects. There is no need to map all your information needs into a well-defined schema, which can be very difficult at times. Imagine a shopping engine. Each category, or even a product group, may need attributes that are a unique combination for the product. So do we create a superset of all attributes, or do we create a keyword-value pair? Or, better still, should we just dump them in a string description and interpret the string at runtime?

ZODB, in practice

ZODB is like a (Python) dictionary. It stores data in a key-value pair, where the value is a pickled (serialised) object. An object could be a container, which is like a dictionary for storing a very large number of elements.

Let us look at a simple example that would be perfectly suitable for a relational database, and see how it may be implemented in ZODB. We have a set of albums, and a set of tracks. Now, you may wish to access the tracks, and from there, if need be, access the album of which it is a part. On the other hand, you may access an album, and then want to access the tracks that make up that album.

In the relational model, you would need a table for each, albums and tracks, and a foreign key from a track to an album. You'd need an additional table to maintain the relationship between the album and tracks. When you realise that a track can be in multiple albums, you'd have to create one more table for that relationship, instead of using a foreign key.

Now, let us look at how to do this using ZODB. The initial step is to create/open the database, open a connection and access its root. Let's write this basic code in *app_db.py*, as you will need to use it in each script that uses the application database.

```
from ZODB import FileStorage, DB
```

```
class app_db(object):
    def __init__(self, path='./data.fs'):
        self.storage = FileStorage.FileStorage(path)
        self.db = DB(self.storage)
        self.connection = self.db.open()
        self.dbrroot = self.connection.root()

    def close(self):
        self.connection.close()
        self.db.close()
        self.storage.close()
```

Let's next write a script, *create_containers.py*, to create b-tree containers for albums and tracks.

```
from app_db import app_db
import transaction
from BTrees.OOBTree import OOBTree
db = app_db()
dbrroot = db.dbrroot
dbrroot['Albums']=OOBTree()
dbrroot['Tracks']=OOBTree()
transaction.commit()
db.close()
```